

agent: discovery-security-agent cli: Claude Code CLI llm: auto run_id: 20260727T125104_oe9yf7 generated_at: 2026-07-27T07:24:13.625Z

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: 2026-07-27 | **Scope:** shende-shweta/FSDKC (branch main) — PHP 8.3 / Laravel 12 backend (backend/), React 19 + TypeScript + Vite 6 frontend (frontend/), and a Node 22 / Express 4 dev/reference API (dev-api/), backed by MariaDB 11 (Eloquent ORM) and MongoDB 7 (native driver). Auth mechanism: none detected — no session, JWT, OAuth, or Sanctum guard on any route.

Executive Summary

EXECUTIVE SUMMARY

This review covered all three layers — the Laravel backend, the React/TypeScript frontend, and the Express dev-api reference server. The dominant, systemic finding is that the entire API surface is unauthenticated and unauthorized: every Discovery, Connect, dashboard, MongoDB, and legacy-report route in both backend/routes/api.php and dev-api/src/server.js is anonymously reachable, and reads/mutations use client-supplied IDs with no ownership checks. On top of that, CORS is wildcard-open (allowed_origins => ['*']) and bare cors(), LegacyReportController runs PHP extract() over raw \$request->all() (variable injection), the dev-api transcript query is exposed to MongoDB operator injection, default database credentials (secret, root) and APP_DEBUG=true are committed to .env.example, docker-compose.yml, and CI, and no rate limiting exists on expensive start / run-check / bulk-import endpoints. The frontend is comparatively clean — React auto-escaping is used throughout, there are no dangerouslySetInnerHTML / innerHTML / eval sinks, no client-side secrets, and no auth tokens in browser storage — its one real gap is the absence of a Content-Security-Policy. Dependencies are current (Laravel 12, React 19.2, Express 4.21, mongodb 6/2) with no known-EOL majors in the manifests. Because at least one unresolved Critical (no authentication) and multiple High findings exist, the overall security rating is High Risk.

24

FILES SCANNED FOR INPUT HANDLING

4

CONCRETE INJECTION/XSS/CSRF/CORS/AUTH FINDINGS

0

DEPENDENCIES FLAGGED OUTDATED/VULNERABLE

7/10

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by a Critical missing-authentication gap across the entire API, wildcard CORS, PHP extract() variable injection, and committed default credentials with debug mode enabled.

6.1 Security Benchmark Ratings

One row per KPI. "Measured" is the real value found; "Rating" is the band it falls into. This table is the source for the Overall Codebase Rating banner above.

No frontend layer coverage gap: a React/TypeScript frontend was detected and reviewed — frontend checks FS1–FS5 are covered in §6.2. **No additional security findings beyond the standard set** were observed outside those documented below.

Dependency KPIs (H6–H8) are assessed from manifest/lockfile inspection only — the review is remote/read-only (GitHub cloud mode), so no live `composer audit` / `npm audit` was executed. The project's own audit note (`docs/CODEBASE_AUDIT_ISSUES.md`) records "0 critical/high at time of audit," consistent with the current manifest versions.

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	1 (unauthenticated API surface)	MODERATE
H2	High Vulnerabilities	0	<5	5–10	>10	3 (CORS, extract() injection, misconfig)	GOOD
H3	Medium Vulnerabilities	low	<20	20–50	>50	5	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	≈3.1/KLOC (9 findings / ≈2.9 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	≈22% categories clean	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	0 (manifest-based; scanner not run)	GOOD
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	<10% (current majors)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	0	GOOD

Overall driver: per the security worst-wins rule, the unresolved Critical (no authentication) plus High-severity CORS/injection/misconfiguration findings force the overall rating to **High Risk**, reinforced by H4 (vulnerability density) and H5 (OWASP compliance) independently landing in the High Risk band.

6.2 Hotspot-by-Hotspot Evidence

Missing Authentication & Authorization (entire API surface) **CRITICAL**

Neither the Laravel API nor the Express `dev-api` registers any authentication or authorization.

`backend/routes/api.php` defines every route with no `middleware('auth:...')` / `->middleware('throttle')`, and controllers resolve resources by client-supplied ID via `findOrFail($id)` with **no ownership or role check** — so any anonymous caller can read all discovery jobs, IVR trees, TFN monitors, MongoDB transcripts, and diagnostics, and can trigger state-changing operations.

`backend/routes/api.php:32-47` — all Discovery/Connect routes are public:

```
Route::prefix('discovery')->group(function (): void {
    Route::get('/jobs', [DiscoveryController::class, 'index']);
    Route::post('/jobs', [DiscoveryController::class, 'store']);
    Route::get('/jobs/{id}', [DiscoveryController::class, 'show']);
    Route::post('/jobs/{id}/start', [DiscoveryController::class, 'start']); //
    no auth, no throttle
});
```

`dev-api/src/server.js:143-161` — an unauthenticated bulk-import that trusts arbitrary client input, including the reachability metric:

```
app.post('/api/connect/monitors/bulk-import', (req, res) => {
    // No validation, no rate limiting – accepts arbitrary body (security audit
    finding)
    const items = Array.isArray(req.body) ? req.body : req.body?.monitors ?? [];
    const created = items.map((item) => { /* trusts item.reachability_pct,
    item.name, ... */ });
});
```

Exploit scenario: An attacker who knows only the base URL sends `GET /api/discovery/jobs/1` and `GET /api/mongodb/transcripts?module=discovery&reference_id=1` to exfiltrate every stored IVR transcript and call diagnostic for any tenant, then `POST /api/connect/monitors/bulk-import` with a large array to inject thousands of fake monitors with forged 100% reachability — all with no credentials.

Recommended fix:

1. Add Laravel Sanctum (`composer require laravel/sanctum`) and wrap all non-public routes in `Route::middleware('auth:sanctum')` in `backend/routes/api.php`; keep only `/health` public.
2. Introduce a `users` → resource ownership model and enforce it with Policies/Gates in `DiscoveryController`, `ConnectController`, `LegacyReportController`, and `MongoController` so `findOrFail($id)` is preceded by `$this->authorize(...)`.
3. In `dev-api/src/server.js`, add an auth middleware (API key/JWT) in front of the router and remove or gate `bulk-import`.
4. Apply `throttle:` middleware (see Insecure Design below) to the same routes.

No files matched `backend/routes/api.php` containing `Route::(get|post|put|patch|delete)` .

No files matched `dev-api/src/**/*.js` containing `app\.(get|post|put|delete)\(\)` .

PHP `extract()` Variable Injection HIGH

`LegacyReportController::carrierSummary()` calls `extract()` directly on unvalidated `$request->all()` , letting any request parameter become a local variable. The same anti-pattern appears in `LegacyDataMapper` .

`backend/app/Http/Controllers/Api/LegacyReportController.php:19-28` :

```
public function carrierSummary(Request $request): JsonResponse
{
    $filters = $request->all();
    extract($filters); // request params → local variables

    $monitors = ConnectMonitor::query()
        ->when(isset($country_code), fn ($q) => $q->where('country_code',
    $country_code))
        ->when(isset($carrier), fn ($q) => $q->where('carrier', $carrier))
        ->orderByDesc('reachability_pct')->get();
```

`backend/app/Legacy/LegacyDataMapper.php:12` — `extract($row, EXTR_SKIP)` and `:23` `extract($context)` (no `EXTR_SKIP` , so caller-controlled keys can clobber locals).

Exploit scenario: A caller sends `GET /api/legacy/reports/carriers?`

`country_code=US&carrier=Verizon&monitors[]=x` ; because `extract()` blindly creates variables from the array, a crafted key such as `?filters=...` or a colliding name can overwrite a local (`$monitors` , `$rows` , `$mapper`) later in scope, corrupting the query result set or the mapper instance and producing unintended filtering/logic.

`LegacyDataMapper::mapJobContext()` uses `extract($context)` with no `EXTR_SKIP` , widening the clobbering surface.

Recommended fix:

1. In `LegacyReportController::carrierSummary` , replace `extract($request->all())` with an explicit `$validated = $request->validate([...])` and read `$validated['country_code']` / `$validated['carrier']` .
2. In `LegacyDataMapper` , remove both `extract()` calls and read array keys explicitly (`$row['name'] ?? 'Unknown'`).
3. Add a static-analysis rule (PHPStan) forbidding `extract()` (the project's `AGENTS.md` already bans it).

No files matched `backend/app/**/*.php` containing `extract\(\)` .

Permissive / Wildcard CORS HIGH

Both APIs allow any origin. The Laravel config uses `*` for origins, methods, and headers; the Express server uses bare `cors()` (also all-origins).

`backend/config/cors.php:4-8` :

```
'paths' => ['api/*', 'up'],
'allowed_methods' => ['*'],
'allowed_origins' => ['*'],
'allowed_headers' => ['*'],
```

`dev-api/src/server.js:17` — `app.use(cors());` (defaults to `Access-Control-Allow-Origin: *`).

Exploit scenario: Any malicious website a victim visits can issue cross-origin `fetch()` calls to the API and read the JSON responses (transcripts, diagnostics, KPIs) in the victim's browser context. Combined with the missing authentication, a drive-by page can silently enumerate and exfiltrate all platform data, or POST new jobs/monitors, from any visitor.

`supports_credentials` is `false`, which limits cookie replay, but the token-less anonymous API means data is exposed regardless.

Recommended fix:

1. In `backend/config/cors.php`, replace `'*'` origins with an explicit allow-list (`env('CORS_ALLOWED_ORIGINS')` parsed to an array) and restrict `allowed_methods` / `allowed_headers` to those actually used.
2. In `dev-api/src/server.js`, configure `cors({ origin: [<trusted origins>], methods: ['GET', 'POST'] })`.
3. Never combine `origin: '*'` with `credentials: true`.

No files matched `backend/config/cors.php`.

Security Misconfiguration — Debug Mode & Committed Default Credentials HIGH

Debug mode is enabled and default database credentials are hard-coded in tracked config, so a production-like deploy from these files ships verbose stack traces and guessable DB passwords.

`backend/.env.example:4` `APP_DEBUG=true`, and `docker-compose.yml:20-26`:

```
APP_ENV: local
APP_DEBUG: "true"
DB_PASSWORD: secret
```

`docker-compose.yml:37-40` and `.github/workflows/ci.yml:15-18` commit `MYSQL_ROOT_PASSWORD: root` and `MYSQL_PASSWORD: secret`. `backend/.env.example:12` also carries `DB_PASSWORD=secret`. `backend/config/app.php:6` correctly defaults `debug` to `false`, but the shipped env/compose overrides it to `true`.

Exploit scenario: If any environment is stood up from `docker-compose.yml` or a copied `.env.example` without hardening, Laravel returns Ignition/Whoops stack traces exposing file paths, SQL, and environment values on any unhandled error; and the MariaDB instance accepts the well-known `klearcom / secret` (and root/ `root`) credentials, allowing direct DB access if port 3306 is reachable.

Recommended fix:

1. Set `APP_DEBUG=false` and `APP_ENV=production` in all non-local env files; keep debug on only in a git-ignored local `.env`.
2. Remove real-looking passwords from `docker-compose.yml`, `.env.example`, and CI; source them from secrets (`${DB_PASSWORD:?}`) / GitHub Actions secrets.
3. Rotate any credential that has ever been committed and add a secret-scanning step to CI.

No files matched `**/*.{yaml,yml}` containing `(APP_DEBUG\s*[:=]\s*\.?true|MYSQL_ROOT_PASSWORD|MYSQL_PASSWORD|DB_PASSWORD\s*[:=]\s*secret)`.

MongoDB Operator Injection (dev-api transcript query) MEDIUM

The Express `dev-api` passes a query-string value straight into a Mongo filter. Express's default `qs` body/query parser turns bracketed keys into nested objects, so a client can inject MongoDB query operators.

`dev-api/src/server.js:36-43` :

```
app.get('/api/mongodb/transcripts', async (req, res) => {
  const { module, reference_id } = req.query;
  if (!module || !reference_id) return res.status(400).json({ error: '...' });
  const data = await getTranscripts(module, Number(reference_id)); // module
  unchecked
});
```

`dev-api/src/mongo.js:117-123` uses it as an equality filter: `.find({ module, reference_id })`.

Exploit scenario: A request like `GET /api/mongodb/transcripts?module[$ne]=xyz&reference_id=1` is parsed by Express into `module = { $ne: 'xyz' }`, so the filter becomes `{ module: { $ne: 'xyz' } }` — returning transcripts for every module and bypassing the intended `module` scoping. Operator injection (`$ne`, `$regex`, `$where` -style payloads on richer queries) enables data enumeration beyond the caller's intended slice. (The Laravel equivalent, `MongoController::transcripts`, is safe because it `validate([... 'module' => 'required|string'])` and coerces with `->string()->toString()`.)

Recommended fix:

1. In `dev-api/src/server.js`, coerce `module` with `String(req.query.module)` before use and reject non-string types.
2. Validate `module` against an allow-list (`['discovery','connect']`) and validate `reference_id` as a positive integer.
3. Add `express-mongo-sanitize` (or equivalent) middleware to strip `$` / `.` keys from `req.query` / `req.body`.

No files matched `dev-api/src/server.js` containing `req\.query\.(\(module|reference_id\)|getTranscripts\(\(module`.

No Rate Limiting on Expensive / Mutating Endpoints MEDIUM

No throttling exists anywhere. `backend/routes/api.php` applies no `throttle` middleware, `backend/bootstrap/app.php` configures no global limiter, and `dev-api/src/server.js` has no rate-limit middleware. Several endpoints kick off background test runs or unbounded writes.

`backend/app/Http/Controllers/Api/DiscoveryController.php:77-79` and `ConnectController.php:93-95` dispatch background jobs on each unauthenticated call; `dev-api/src/server.js:143-161` `bulk-import` accepts an arbitrary-length array with no size cap.

Exploit scenario: An attacker scripts thousands of `POST /api/discovery/jobs/{id}/start` and `POST /api/connect/monitors/{id}/run-check` requests, or one `bulk-import` with a multi-megabyte array, exhausting queue workers / process memory and denying service — no credentials or per-IP limit stands in the way.

Recommended fix:

1. Add `->middleware('throttle:60,1')` (or a named limiter in `bootstrap/app.php`) to write/expensive routes in `backend/routes/api.php`.
2. Add `express-rate-limit` to `dev-api/src/server.js` and a body-size limit (`express.json({ limit: '100kb' })`), plus a max-items cap on `bulk-import`.

No files matched `backend/app/Http/Controllers/Api/**/*.php` containing `(dispatch\(|afterResponse\(|\)|bulk-import)`.

Missing Security Headers & CSP MEDIUM

No security headers are set at the edge or the app. `docker/nginx/default.conf` adds no `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, or `X-Content-Type-Options`, and `frontend/index.html` has no CSP `<meta>`.

`docker/nginx/default.conf:1-17` (no `add_header` directives); `frontend/index.html:1-15` (no CSP meta).

Exploit scenario: Without CSP/ `X-Frame-Options`, the SPA can be framed for clickjacking and has no defence-in-depth against injected script; without HSTS, a downgraded connection can be MITM'd. Impact is bounded because the frontend itself has no XSS sink today, but the headers are the standard mitigation layer and are entirely absent.

Recommended fix:

1. Add `add_header` directives for CSP, HSTS, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Referrer-Policy` in `docker/nginx/default.conf`.
2. Add a restrictive CSP `<meta http-equiv>` (or serve it as a header) covering the Google Fonts origins used in `frontend/index.html`.

No files matched `docker/nginx/default.conf`.

Frontend security checks (FS1–FS5). A React 19 + TypeScript SPA is present under `frontend/src/`, so the named frontend checks were run explicitly:

FS1 — DOM/Stored/Reflected XSS Sinks CLEAN

Evidence: Not observed — a repo-wide grep for `dangerouslySetInnerHTML`, `innerHTML`, `outerHTML`, `document.write`, `eval()`, and `new Function()` across `frontend/` and `dev-api/` returned zero matches. All dynamic values render through JSX text interpolation, which React auto-escapes — e.g.

`frontend/src/pages/DiscoveryPage.tsx:168` renders untrusted transcript data as `{String(t.payload?.transcript ?? JSON.stringify(t.payload))}` (escaped text, not markup), and

`frontend/src/components/IvrTree.tsx:28` renders `{node.prompt_text}` as text. No directive emitted (no vulnerable sink to target).

FS2 — Secrets / API Keys in Client Code CLEAN

Evidence: Not observed — the only build-time variable read in client code is `import.meta.env.VITE_API_URL` (`frontend/src/api/client.ts:1`), which is a non-secret base URL. No `apiKey`, `secret`, `Bearer`, `sk-`, `ghp_`, `AIza`, or private-key material was found in `frontend/`. No directive emitted.

FS3 — Auth Tokens in Browser Storage CLEAN

Evidence: Not observed — no `localStorage` / `sessionStorage` usage exists anywhere in `frontend/`, and there is no auth token to store because the API is token-less/anonymous (see the Critical auth finding). If authentication is added, tokens must be kept out of JS-readable storage (prefer HttpOnly cookies). No directive emitted.

FS4 — Vulnerable / Outdated npm Dependencies CLEAN

Evidence: Not observed at manifest level — `frontend/package.json` pins current majors (`react@^19.2.3`, `react-dom@^19.2.3`, `react-router-dom@^7.1.0`, `@tanstack/react-query@^5.62.0`, `vite@^6.0.3`, `typescript@^5.7.2`); `dev-api/package.json` pins `express@^4.21.2`, `mongodb@^6.12.0`, `cors@^2.8.5`. None are known-EOL majors. A live `npm audit` was not executed (remote read-only review); wire it into CI to keep this continuously verified. No directive emitted.

FS5 — Missing Frontend Security Controls MEDIUM

Evidence: `frontend/index.html:1-15` ships no Content-Security-Policy `<meta>` and no other frontend hardening, and `frontend/src/api/client.ts:1` defaults the API base to a non-TLS `http://localhost:8080/api`. No insecure `postMessage`, `target="_blank"`, or client-side-only authorization was found (grep clean). This is a real but bounded frontend gap — the missing CSP is the same control called out server-side under Security Misconfiguration.

Exploit scenario: With no CSP delivered to the browser, there is no second line of defence if a future change introduces an injection sink, and the page can be framed for clickjacking; the `http://` default base also permits a downgrade/MITM in a misconfigured deploy.

Recommended fix: Add a restrictive CSP (`default-src 'self'`, explicit font/style origins for Google Fonts) via `<meta http-equiv>` in `frontend/index.html` or as a served header, and make the API base HTTPS-only in `frontend/src/api/client.ts`.

No files matched `frontend/index.html`.

Not observed hotspots:

- **SQL Injection** — Not observed. The Laravel backend uses Eloquent exclusively; a grep for `DB::raw`, `whereRaw`, `selectRaw`, `DB::select`, and `DB::statement` across `backend/` returned zero matches. All parameters bind through the query builder.
- **Server-Side Request Forgery (SSRF)** — Not observed. No server-side HTTP fetch is built from user-supplied URLs; the "test call" flows in `RealTimeTestService` / `dev-api/src/realtime.js` are simulated (`usleep` / `Math.random`), and MongoDB connects only to the configured URI.

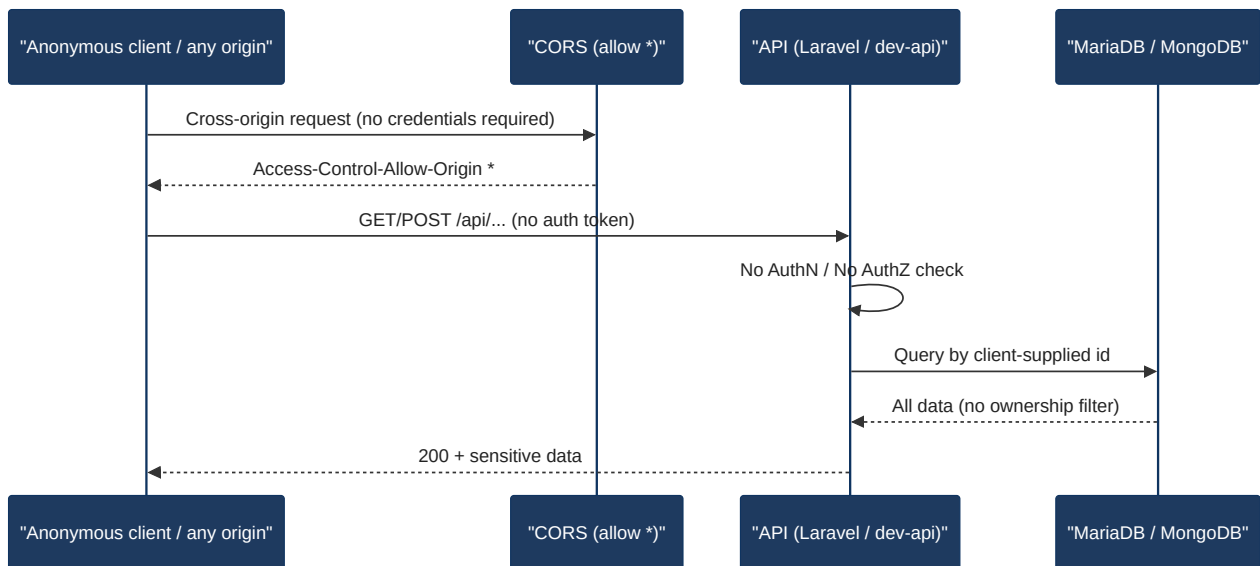
- **Insecure Deserialization / unsafe file upload / path traversal** — Not observed. No `unserialize()`, no file-upload handlers, and no user-controlled filesystem paths were found.

6.3 OWASP Top 10 (2021) Coverage

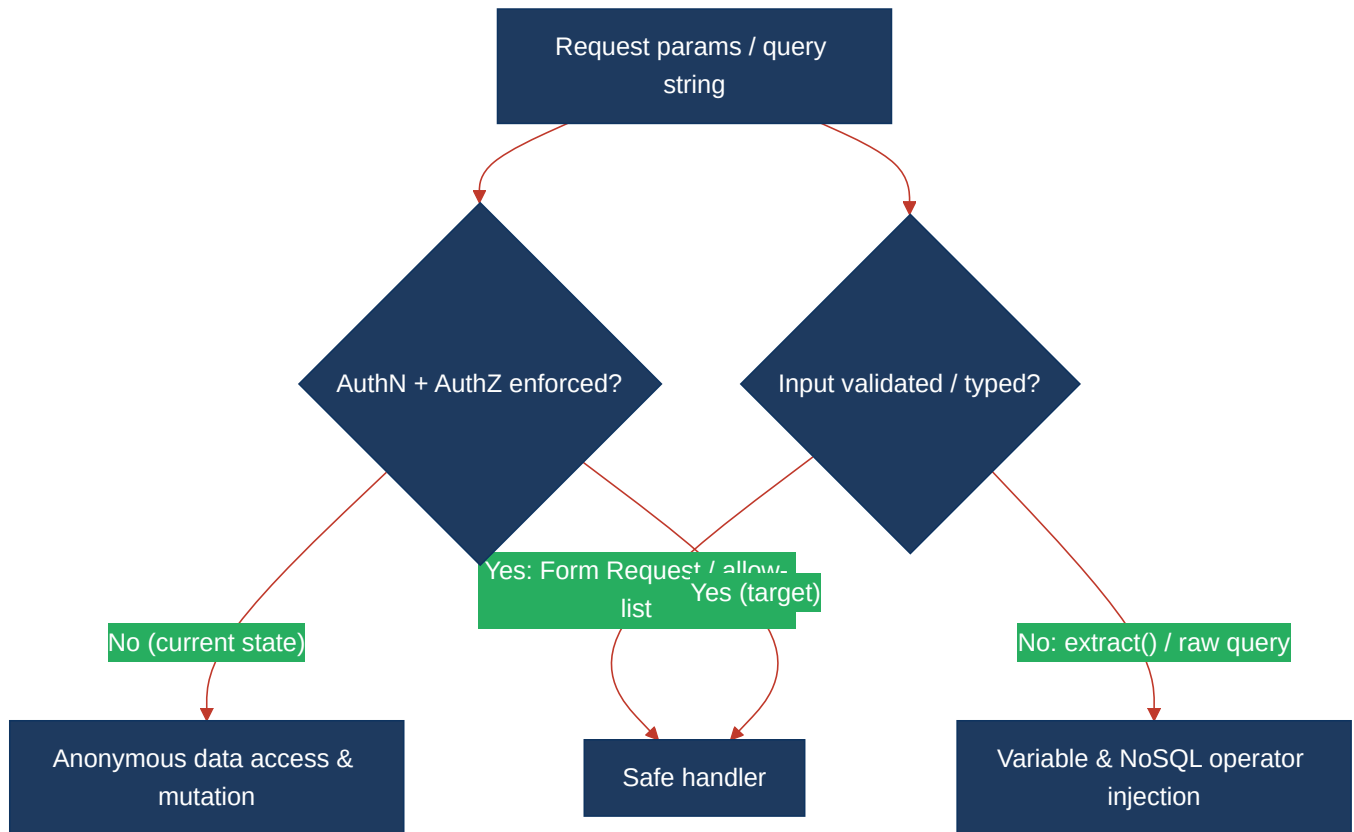
#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	CRITICAL	No auth/authorization on any route (<code>backend/routes/api.php</code> , <code>dev-api/src/server.js</code>); <code>findOrFail(\$id)</code> with no ownership check; unauthenticated <code>bulk-import</code> .
6.2	Cryptographic Failures	HIGH	Default DB passwords (<code>secret</code> / <code>root</code>) committed to <code>docker-compose.yml</code> , <code>.env.example</code> , CI; empty <code>APP_KEY</code> in <code>.env.example</code> . No user passwords are stored (no accounts), so no hashing weakness.
6.3	Injection	HIGH	PHP <code>extract()</code> variable injection in <code>LegacyReportController</code> / <code>LegacyDataMapper</code> ; MongoDB operator injection in <code>dev-api</code> transcripts query. No SQLi (Eloquent) and no frontend XSS sinks (FS1).
6.4	Insecure Design	MEDIUM	No rate limiting/lockout; <code>bulk-import</code> trusts client-supplied <code>reachability_pct</code> and unbounded array size.
6.5	Security Misconfiguration	HIGH	<code>APP_DEBUG=true</code> in shipped env/compose; wildcard CORS; no security headers/CSP (nginx + <code>index.html</code> , FS5).
6.6	Vulnerable & Outdated Components	CLEAN	Manifests pin current majors (Laravel 12, React 19.2, Express 4.21, mongodb 6/2); no EOL majors. Live scanner not run remotely — wire into CI.
6.7	Identification & Authentication Failures	CRITICAL	No authentication mechanism at all — no login, sessions, JWT, MFA, or lockout on any endpoint.
6.8	Software & Data Integrity Failures	CLEAN	Lockfiles present (<code>package-lock.json</code>); minor: CI actions pinned to major tag not SHA, <code>pecl install mongodb</code> unpinned. No insecure deserialization.
6.9	Security Logging & Monitoring Failures	MEDIUM	No audit logging of access/auth events; <code>dev-api</code> uses <code>console.error</code> only; no alerting.
6.10	Server-Side Request Forgery (SSRF)	Not applicable	No server-side HTTP calls built from user-supplied URLs; test flows are simulated.
6.11	Other Security Reviews	CLEAN	No SQLi, no file upload, no path traversal, no <code>unserialize()</code> , no mass-assignment beyond validated <code>store()</code> DTOs.
6.12	DevSecOps Security Assessment	HIGH	Secrets committed to CI/compose; no SAST or dependency-scan step in <code>.github/workflows/ci.yml</code> ; no branch-protection or secret-scanning evidence.

6.4 Diagrams

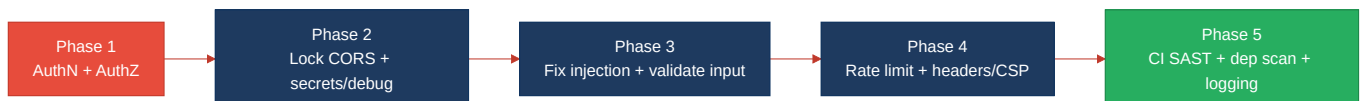
Auth / request trust boundary



Top security risk flow



Improvement roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
Missing authentication & authorization (whole API)	Add Sanctum/JWT auth + ownership policies to all non-public routes in both APIs; gate/remove <code>bulk-import</code>	HIGH RISK	CRITICAL
Identification & Authentication Failures (6.7)	Introduce login, session/JWT lifecycle, MFA option, and brute-force lockout	HIGH RISK	CRITICAL
PHP <code>extract()</code> variable injection (6.3)	Replace <code>extract()</code> with validated explicit access in <code>LegacyReportController</code> & <code>LegacyDataMapper</code> ; ban via PHPStan	MODERATE	HIGH
Wildcard CORS (6.5)	Replace <code>*</code> with explicit origin allow-list in <code>config/cors.php</code> and <code>dev-api cors()</code>	MODERATE	HIGH
Debug mode + committed default credentials (6.2/6.5)	Disable debug in prod, move secrets to a secret store, rotate exposed creds	MODERATE	HIGH
MongoDB operator injection (6.3)	Coerce/allow-list <code>module</code> , add <code>express-mongo-sanitize</code> in <code>dev-api</code>	MODERATE	MEDIUM
No rate limiting (6.4)	Add <code>throttle</code> (Laravel) / <code>express-rate-limit</code> + body-size caps	MODERATE	MEDIUM
Missing security headers & CSP (6.5 / FS5)	Add CSP/HSTS/X-Frame-Options at nginx edge and a CSP in <code>index.html</code>	MODERATE	MEDIUM
No security logging/monitoring (6.9)	Add structured audit logging for access/auth events + alerting	MODERATE	MEDIUM
No SAST/dependency scan in CI (6.12)	Add <code>composer audit</code> + <code>npm audit</code> + a SAST step and secret scanning to <code>ci.yml</code>	MODERATE	MEDIUM

6.6 Expected Outcomes

- Enforcing authentication and per-resource authorization eliminates the anonymous read/write path to all Discovery, Connect, transcript, and diagnostic data — closing the dominant account-takeover / data-exfiltration risk.
- Locking CORS to a trusted origin allow-list stops malicious cross-origin pages from reading API responses in a victim's browser.
- Removing `extract()` and validating/allow-listing MongoDB query inputs eliminates the PHP variable-injection and NoSQL operator-injection vectors.
- Disabling debug in production, rotating committed credentials, and adding security headers/CSP remove information-disclosure and clickjacking/defence-in-depth gaps across backend and frontend.
- Wiring `composer audit` / `npm audit` and a SAST step into CI turns dependency and code-security regressions into automatic, blocking checks so future CVEs are caught before release.